

Sahithi Macharapu
DA24B024

Q1.

Single naive version - 2.21 GB

Multistage version - 671 MB

<code>multistage-image:latest</code>	6771ab4015b9	671MB	144MB
<code>naive-image:latest</code>	726198f8b70a	2.21GB	572MB

I used the slim version and the normal version of python in the multistage case, but only used the normal version of python in the single version. The multistage version I used only had the api calls at the end, the dataset, training and model were found at the intermediate steps and only the model was saved, so the memory requirement was low.

Q2.

```
(myenv) vboxuser@Ubuntu:~/Downloads/AiOpsAssign2$ time curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text":"WIN a FREE iPhone now! Click here: bit.ab/a123"}'
{"label":"spam"}
real    0m0.032s
user    0m0.001s
sys     0m0.005s
(myenv) vboxuser@Ubuntu:~/Downloads/AiOpsAssign2$ time curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text":"WIN a FREE iPhone now! Click here: bit.ab/a123"}'
{"label":"spam"}
real    0m0.015s
user    0m0.002s
sys     0m0.009s
```

Docker Compose makes it easier to run and define different containers that are related, which is something a single Dockerfile cannot handle. Kubernetes is used for a bigger scale where you have containers spread across multiple machines while Compose is usually for multi-container environments.

Q3.

- I used 4 for parallelisation because it's a factor of 8 and would divide the work efficiently so no machine is left idle at any time.
- I decided to use higher values for the limits as compared to the requests and smaller values like 16 seemed too small to save a whole row, so I used 64 and 128.
- If we use a shared container, there is a chance of it getting corrupted, so it would be better to use the API to read the logs.
- Yes, it would be better to change it to 6 because that would be a factor of 6 and since 8 can't divide itself amongst 6 CPUs, this would give the best throughput rate. This is

because 6 of them can be done in the first round and 2 in the second, so 4 of them will be free to be used for other things.

```
(myenv) vboxuser@Ubuntu:~/Downloads/Assign2/Q3$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP
IP   OS-IMAGE          KERNEL-VERSION   CONTAINER-RUNTIME
minikube            Ready     control-plane  80s   v1.35.1   192.168.49.2   <none>
                Debian GNU/Linux 12 (bookworm)  7.0.0-31-generic  docker://29.2.1
minikube-m02        Ready     <none>        64s   v1.35.1   192.168.49.3   <none>
                Debian GNU/Linux 12 (bookworm)  7.0.0-31-generic  docker://29.2.1
(myenv) vboxuser@Ubuntu:~/Downloads/Assign2/Q3$
```

```
(myenv) vboxuser@Ubuntu:~/Downloads/Assign2/Q3$ cat shards/ground_truth.json
{"shard_0": 1, "shard_1": 4, "shard_2": 2, "shard_3": 2, "shard_4": 2, "shard_5": 1, "shard_6": 1, "shard_7": 2}(myenv) vboxuser@Ubuntu:~/Downloads/Assign2/Q3$
```

The final shards:

```
{"shard_0": 1, "shard_1": 4, "shard_2": 2, "shard_3": 2, "shard_4": 2, "shard_5": 1, "shard_6": 1, "shard_7": 2}
{"shard_0": 1, "shard_1": 4, "shard_2": 2, "shard_3": 2, "shard_4": 2, "shard_5": 1, "shard_6": 1, "shard_7": 2}
```

We can see that the values match, so the API works.

Q4.

- The ReplicaSet is responsible. It runs continuously comparing its desired replica count against the number of Pods actually matching its label selector.
- This can be seen through the way kubectl gets pods showing a new pod name and reset age appears within seconds of the delete.
- A Job has no ReplicaSet, so it has no rolling-update strategy. Getting v2 would mean that we would have had to delete and recreate the whole Job with the new image.
- The Deployment's ReplicaSet continuously moves towards a target replica count and its RollingUpdate strategy replaces Pods gradually which is what is required by a long-lived serving workload.
- But on the other hand, Job runs each Pod to completion and stops, so for cases like hyperparameter search, this would be perfect.